

MADRL 算法完整讲解

从零基础强化学习到项目中的 MATD3、reward function 和 safety
projection

MADRL_ESS 项目理解笔记

2026 年 7 月 27 日

目录

1	这份文档怎么读	4
2	强化学习最基本的故事	4
2.1	强化学习不是“直接给答案”	4
2.2	一个时间步发生了什么	5
3	状态、观测、动作、奖励	5
3.1	状态和观测	5
3.2	动作	6
3.3	奖励	6
4	为什么不能只看当前奖励	7
4.1	当前省钱不一定长期好	7
4.2	return 是什么	7
5	Q 值: critic 在估计什么	7
5.1	Q 值的通俗解释	7
5.2	为什么 critic 需要看联合动作	8
6	Bellman 方程和 Bellman target	8
6.1	Bellman 方程的直觉	8
6.2	Bellman target 是什么	8
6.3	本项目为什么用 n-step target	9
6.4	bootstrap 是什么	9

目录	2
7 Actor 和 critic	9
7.1 Actor: 负责做动作	9
7.2 Critic: 负责评价动作	10
7.3 Actor 怎么学习	10
8 Replay buffer: 为什么要存经验	10
8.1 transition 是什么	10
8.2 为什么不能边走边只学最新一步	11
9 TD3 和 MATD3: 项目算法骨架	11
9.1 TD3 是什么	11
9.2 Target network: 慢一点的老师	11
9.3 Target policy smoothing	12
9.4 Delayed actor update	12
10 集中训练、分散执行 CTDE	12
10.1 为什么需要 CTDE	12
10.2 CTDE 的好处	12
11 项目中的观测输入	13
11.1 Actor 输入	13
11.2 Critic 输入	13
12 项目中的动作后处理	13
12.1 为什么 raw action 不能直接执行	13
12.2 电池动作映射	14
13 PV 在项目中如何处理	14
13.1 PV 是预测输入, 不是 actor 创造出来的	14
13.2 PV action 到利用率	14
13.3 PV 如何影响净负荷	15
13.4 PV 有没有直接 reward 惩罚	15
14 项目 reward function 逐项解释	15
14.1 总公式	15
14.2 电池收益	15
14.3 EV 充电成本	16
14.4 动作边界惩罚	16
14.5 SoC 正则惩罚	16
14.6 EV SoC、离家、进度和价格惩罚	16

14.7 throughput bonus	17
14.8 电网安全惩罚	17
15 项目训练流程	17
15.1 整体伪代码	17
15.2 探索噪声	18
15.3 Critic 更新流程	18
15.4 Actor 更新流程	19
16 Safety penalty 和 safety projection	19
16.1 Safety penalty: 事后扣分	19
16.2 Safety projection: 执行前修正	19
17 Safety projection 如何实现	20
17.1 投影变量	20
17.2 为什么电池和 PV 弃光可以一起投影	20
17.3 灵敏度矩阵	20
17.4 基础净负荷	20
17.5 约束写成 half-space	21
17.6 逐行投影公式	21
17.7 投影后的动作转回 actor 格式	21
17.8 projection 的代价	21
18 执行阶段: 训练好以后用什么	22
19 从代码角度看完整闭环	22
20 常见名词小词典	23
21 总结	23

1 这份文档怎么读

本文件默认读者没有学过 DRL，也没有学过 MADRL。因此前几节不会直接讲代码，而是先回答一些最基础的问题：

- 什么是强化学习；
- 什么是状态、观测、动作、奖励；
- 为什么要关心未来累计奖励；
- Q 值、Bellman target、critic、actor 到底是什么意思；
- replay buffer、target network、twin critic 为什么存在。

后半部分再回到本项目，讲清楚：

- 项目里的 MADRL 训练流程；
- actor 和 critic 的输入输出；
- reward function 的每一项；
- PV 在这里如何作为预测输入和可控弃光变量进入净负荷；
- safety penalty 和 safety projection 的区别；
- MADRL_BASE、MADRL_PENALTY、MADRL_PROJECTION 三个方案的差异。

EV 和静态电池的详细物理建模已经在前面的文档中讲过，这里只在解释动作、reward 和 projection 时简要使用。

2 强化学习最基本的故事

2.1 强化学习不是“直接给答案”

监督学习里，我们通常给模型很多样本，例如“这张图是猫”“这张图是狗”，模型学习输入到标签的对应关系。

强化学习不同。强化学习没有每一步的标准答案。它更像训练一个会试错的控制器：

看见当前情况 → 做一个动作 → 环境发生变化 → 得到奖励或惩罚。 (1)

控制器一开始可能乱做。它通过大量试错慢慢学会：哪些动作长期来看更好，哪些动作虽然眼前不错但以后会出问题。

在本项目中，这个故事对应为：

- 环境：低压配电网、家庭负荷、PV、静态电池、EV；

- 智能体 agent：每个 prosumer 的控制器；
- 动作：电池充放电、PV 使用/弃光、EV 充电；
- 奖励：电费收益、EV 成本、SoC 惩罚、电压/线路/变压器安全惩罚等。

2.2 一个时间步发生了什么

强化学习通常按离散时间步运行。本项目里一个时间步是：

$$\Delta t = 0.25 \text{ 小时} = 15 \text{ 分钟}. \quad (2)$$

在时间步 t ，闭环是：

1. 环境给出观测 o_t ；
2. actor 根据观测输出动作 a_t ；
3. 动作经过本地物理裁剪和可选 safety projection；
4. 环境执行动作，更新电池 SoC、EV SoC、PV 利用、净负荷；
5. pandapower 计算潮流，得到电压、线路负载率、变压器负载率；
6. 环境计算 reward；
7. 进入下一步 $t + 1$ 。

写成公式就是：

$$o_t \xrightarrow{\pi} a_t \xrightarrow{\text{env.step}} (r_t, o_{t+1}). \quad (3)$$

3 状态、观测、动作、奖励

3.1 状态和观测

理论上，状态 s_t 表示环境在时刻 t 的完整真实情况。如果知道完整状态，就等于知道系统现在的一切。

但实际控制器通常看不到所有信息，所以项目里更重要的是观测 o_t 。观测是 agent 能看到、能输入神经网络的信息。

对第 i 个 agent：

$$o_{i,t} = \text{agent } i \text{ 在 } t \text{ 时刻看到的信息}. \quad (4)$$

本项目的 actor 观测包括：

- 时间周期特征，例如一天中的时间、一年中的日期；
- 静态电池 SoC；

- EV SoC;
- EV 是否连接;
- 未来价格窗口;
- 未来负荷预测;
- 未来 PV 预测。

3.2 动作

每个 agent 的动作是三维连续动作:

$$a_{i,t} = [a_{i,t}^{bat}, a_{i,t}^{pv}, a_{i,t}^{ev}], \quad a_{i,t} \in [-1, 1]^3. \quad (5)$$

三个维度含义是:

- a^{bat} : 电池动作, 之后会映射成电池功率;
- a^{pv} : PV 动作, 之后会映射成 PV 利用率;
- a^{ev} : EV 动作, 之后会映射成 EV 充电功率。

注意: actor 输出的不是直接 kW, 而是归一化动作。这样神经网络永远只需要输出 $[-1, 1]$ 范围内的数, 物理单位转换由环境和后处理完成。

3.3 奖励

奖励 $r_{i,t}$ 是 agent 学习的方向。奖励越高, 说明这个动作越值得保留; 奖励越低, 说明这个动作带来成本或风险。

本项目的 reward 不是单纯电费, 而是多项组合。代码中的总 reward 可以概括为:

$$\begin{aligned} r_{i,t} = & R_{i,t}^{stor} + R_{i,t}^{ev} - L_{i,t}^{act} - L_{i,t}^{soc} - L_{i,t}^{ev,soc} \\ & - L_{i,t}^{ev,dep} - L_{i,t}^{ev,over} - L_{i,t}^{ev,prog} - L_{i,t}^{ev,price} \\ & - L_{i,t}^{ev,proj} - L_{i,t}^{ev,eme} + B_{i,t}^{through} \\ & - L_{i,t}^v - L_{i,t}^\ell - L_{i,t}^{tr}. \end{aligned} \quad (6)$$

这里第一行有一个容易漏看的点: $R_{i,t}^{ev}$ 本身是负的 EV 充电成本, 所以它在公式中以加号出现。更直观地说:

$$R_{i,t}^{ev} = -C_{i,t}^{ev}. \quad (7)$$

4 为什么不能只看当前奖励

4.1 当前省钱不一定长期好

如果 agent 只追求当前 15 分钟奖励，它可能学出很短视的策略。例如：

- 当前电价高，所以 EV 不充电；
- 但一直拖到离家前，EV SoC 不够；
- 最后被离家惩罚或 hard projection 强制高价充电。

因此强化学习要关心未来累计奖励，叫 return。

4.2 return 是什么

从时刻 t 开始，未来总回报为：

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \dots \quad (8)$$

γ 叫折扣因子，范围是：

$$0 \leq \gamma \leq 1. \quad (9)$$

如果 $\gamma = 0$ ，算法只看当前奖励。如果 γ 接近 1，算法会很重视未来。本项目默认：

$$\gamma = 0.999. \quad (10)$$

这说明能源调度需要长视野，因为电池 SoC、EV 离家需求和电网峰值都不是一步动作能解释完的。

5 Q 值：critic 在估计什么

5.1 Q 值的通俗解释

Q 值可以理解成：

如果我现在在这种情况下做这个动作，从现在到未来总共大概能拿多少分。

数学上：

$$Q^\pi(s_t, a_t) = \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \mid s_t, a_t, \pi \right]. \quad (11)$$

这里 π 是之后继续使用的策略。 $Q^\pi(s_t, a_t)$ 不是只评价当前动作的即时奖励，而是评价“当前动作加上后续策略”带来的长期价值。

如果把它说得更生活化：

Q 值不是问“这一口饭好不好吃”，而是问“这样吃下去，整顿饭体验会不会好”。

5.2 为什么 critic 需要看联合动作

本项目是多智能体电网系统。某个 agent 的动作会影响公共电网状态：

$$\mathbf{a}_t = (a_{1,t}, a_{2,t}, \dots, a_{N,t}) \rightarrow \text{潮流} \rightarrow \text{安全惩罚}. \quad (12)$$

因此项目中的 critic 不是只看 agent 自己，而是看联合观测和联合动作：

$$Q_i(\mathbf{o}_t, \mathbf{a}_t). \quad (13)$$

其中：

$$\mathbf{o}_t = (o_{1,t}, o_{2,t}, \dots, o_{N,t}), \quad \mathbf{a}_t = (a_{1,t}, a_{2,t}, \dots, a_{N,t}). \quad (14)$$

这让 critic 能学习“大家同时这样做”对第 i 个 agent reward 的长期影响。

6 Bellman 方程和 Bellman target

6.1 Bellman 方程的直觉

Bellman 方程是强化学习里最核心的想法之一。它说：

当前动作的长期价值 = 当前这一步奖励 + 下一步继续行动的长期价值。

公式是：

$$Q(s_t, a_t) \approx r_t + \gamma Q(s_{t+1}, a_{t+1}). \quad (15)$$

这句话非常重要。它把一个很长的未来问题，拆成了“眼前一步 + 剩下未来”。

6.2 Bellman target 是什么

critic 是神经网络，它会输出一个预测值：

$$\hat{Q} = Q_{\phi}(s_t, a_t). \quad (16)$$

但训练神经网络需要一个“目标答案”。Bellman target 就是用 Bellman 方程临时构造出来的目标答案：

$$y_t = r_t + \gamma Q_{\phi^-}(s_{t+1}, a_{t+1}). \quad (17)$$

这里 ϕ^- 表示 target critic 的参数。target critic 是一个变化较慢的 critic，用它生成目标值会更稳定。

critic 的训练目标是让自己的预测接近 Bellman target：

$$L^{critic} = (Q_{\phi}(s_t, a_t) - y_t)^2. \quad (18)$$

所以 Bellman target 可以理解为：

用“已经拿到的当前奖励”和“对未来的旧网络估计”拼出来的训练标签。

6.3 本项目为什么用 n-step target

如果只用一步 target:

$$y_t = r_t + \gamma Q(s_{t+1}, a_{t+1}), \quad (19)$$

critic 每次只直接看到一步 reward，长期信号传播比较慢。

本项目使用 n-step return。先把未来 n 步真实 reward 加起来，再接上未来 Q 估计：

$$y_{i,t} = \sum_{k=0}^{n-1} \gamma^k r_{i,t+k} + \gamma^n \min(Q_{i,1}^{target}(\mathbf{o}_{t+n}, \mathbf{a}'_{t+n}), Q_{i,2}^{target}(\mathbf{o}_{t+n}, \mathbf{a}'_{t+n})). \quad (20)$$

代码中 $n = \text{cfg.train.n_step_return}$ ，默认是 48。每步 15 分钟，所以 48 步约等于 12 小时。这对 EV 夜间充电很有意义，因为晚上是否充电会影响早晨离家。

6.4 bootstrap 是什么

上式最后的 Q 值叫 bootstrap 项。bootstrap 的意思是：

真实 reward 只累加到某一步，后面太远的未来不直接展开，而是用 critic 的估计接上。

在代码的 `ReplayBuffer` 中，n-step transition 会保存：

- n-step reward;
- next observation;
- bootstrap discount。

如果 episode 已结束，后面没有未来了，bootstrap discount 就是 0；如果 episode 没结束，bootstrap discount 就是 γ^n 。

7 Actor 和 critic

7.1 Actor：负责做动作

Actor 是策略网络。它把观测映射为动作：

$$a_{i,t} = \pi_{\theta_i}(\mathbf{o}_{i,t}). \quad (21)$$

项目中每个 agent 有一个自己的 actor。代码在 `models/assembly.py` 的 `Actor` 类中定义，结构是：

$$\text{输入} \rightarrow \text{Linear} \rightarrow \text{ReLU} \rightarrow \text{Linear} \rightarrow \text{ReLU} \rightarrow \text{Linear} \rightarrow \tanh. \quad (22)$$

最后的 $\tanh$ 把输出压到 $[-1, 1]$ 。

7.2 Critic: 负责评价动作

Critic 是价值网络。它输入联合观测和联合动作，输出 Q 值：

$$Q_{i,\phi_i}(\mathbf{o}_t, \mathbf{a}_t). \quad (23)$$

项目中每个 agent 有自己的 critic，而且是 twin critic：

$$Q_{i,1}, \quad Q_{i,2}. \quad (24)$$

twin critic 的作用是降低 Q 值过高估计。target Q 取二者较小值：

$$\min(Q_{i,1}^{target}, Q_{i,2}^{target}). \quad (25)$$

如果只用一个 critic，actor 可能会利用 critic 的错误高估，选择看似价值很高但实际不好的动作。取较小值是一种保守策略。

7.3 Actor 怎么学习

Actor 的目标是选择 critic 认为长期价值更高的动作：

$$\max_{\theta_i} Q_{i,1}(\mathbf{o}_t, \mathbf{a}_t^{policy}). \quad (26)$$

代码里通常通过最小化负值实现：

$$L_i^{actor} = -\mathbb{E} \left[Q_{i,1}(\mathbf{o}_t, \mathbf{a}_t^{policy}) \right]. \quad (27)$$

所以可以把训练关系理解为：

critic 学会打分，actor 学会拿高分。

8 Replay buffer: 为什么要存经验

8.1 transition 是什么

一次交互会产生一条经验，叫 transition：

$$(\mathbf{o}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{o}_{t+1}, done). \quad (28)$$

它记录了：当时看到了什么、做了什么、拿到什么 reward、下一步看到了什么、episode 是否结束。

8.2 为什么不能边走边只学最新一步

如果只用最新一步训练，数据强相关，训练容易抖动。例如一天中的相邻 15 分钟非常相似，神经网络连续看到高度相似样本，可能学得不稳。

Replay buffer 是经验池。项目把很多 transition 存起来，训练时随机抽样：

$$\mathcal{B} \sim \text{ReplayBuffer}. \quad (29)$$

好处是：

- 打乱时间相关性；
- 旧经验可以重复使用；
- batch 训练更稳定。

9 TD3 和 MATD3: 项目算法骨架

9.1 TD3 是什么

TD3 是一种适合连续动作空间的 actor-critic 算法。它有三个核心技巧：

- twin critic: 两个 critic 取较小值，降低过高估计；
- target policy smoothing: target action 加一点小噪声，避免 critic 对某个精确动作点过拟合；
- delayed actor update: critic 先多学几步，actor 不要太频繁更新。

本项目的主线接近 MATD3，也就是多智能体版本的 TD3。

9.2 Target network: 慢一点的老师

项目中 actor 和 critic 都有 target network:

$$\pi_{\theta_i}^{target}, \quad Q_{\phi_i}^{target}. \quad (30)$$

它们不是每一步直接复制在线网络，而是慢慢更新：

$$\theta^{target} \leftarrow (1 - \tau)\theta^{target} + \tau\theta. \quad (31)$$

项目默认：

$$\tau = 0.01. \quad (32)$$

这叫 soft update。直觉上，target network 是一个慢一点的老师，避免训练目标变化太快。

9.3 Target policy smoothing

计算下一步 target action 时，项目先用 target actor 生成动作，再加噪声：

$$\mathbf{a}'_{t+n} = \text{clip}(\pi^{\text{target}}(\mathbf{o}_{t+n}) + \epsilon, -1, 1). \quad (33)$$

噪声也会被裁剪：

$$\epsilon \sim \mathcal{N}(0, \sigma^2), \quad \epsilon \in [-c, c]. \quad (34)$$

项目中：

$$\text{policy_noise} = 0.2, \quad \text{noise_clip} = 0.5. \quad (35)$$

这个技巧的意思是：critic 不应该只在某个非常精确的动作点上给高分，而应该在动作附近也比较平滑。

9.4 Delayed actor update

项目不是每次 critic 更新都更新 actor，而是每隔若干次更新 actor：

$$\text{policy_update_freq} = 2. \quad (36)$$

原因是 actor 依赖 critic 的评价。如果 critic 还没学准，actor 太早跟着 critic 走，可能越学越偏。让 critic 先多学一点，actor 再更新，通常更稳定。

10 集中训练、分散执行 CTDE

10.1 为什么需要 CTDE

本项目有多个 agent。执行时，每个家庭不一定能看到全网所有信息，所以 actor 分散执行：

$$\mathbf{a}_{i,t} = \pi_{\theta_i}(\mathbf{o}_{i,t}). \quad (37)$$

但训练时，critic 可以看到联合观测和联合动作：

$$Q_i(\mathbf{o}_t, \mathbf{a}_t). \quad (38)$$

这叫 centralized training, decentralized execution, 简称 CTDE。

10.2 CTDE 的好处

CTDE 的好处是：

- 执行时简单，每个 agent 只用自己的 actor；
- 训练时更聪明，critic 能看到多 agent 的相互影响；
- 适合电网这类耦合系统，因为一个用户动作会影响公共电压和变压器。

11 项目中的观测输入

11.1 Actor 输入

代码中 actor 输入由 `actor_features` 拼接：

$$o_{i,t}^{actor} = \left[o_{i,t}^{local}, \rho_{t:t+H-1}^p, \sigma_{t:t+H-1}^p, \hat{L}_{i,t:t+H-1}, \hat{G}_{i,t:t+H-1} \right]. \quad (39)$$

各部分含义：

- o^{local} ：本地状态，包含时间周期特征、电池 SoC、EV SoC、EV 是否连接；
- ρ^p ：未来价格在窗口中的相对高低；
- σ^p ：未来价格窗口的价差强度；
- $\hat{L}$ ：未来负荷预测；
- $\hat{G}$ ：未来 PV 预测。

代码中 actor 输入维度是：

$$7 + 4H. \quad (40)$$

$H = \text{cfg.obs.sequence_length}$ ，默认是 49。

11.2 Critic 输入

critic 输入由 `critic_features` 拼接：

$$o_t^{critic} = \left[o_t^{local}, \rho_{t:t+H-1}^p, \sigma_{t:t+H-1}^p, \hat{L}_{t:t+H-1}, \hat{G}_{t:t+H-1}, \mathbf{a}_t \right]. \quad (41)$$

它比 actor 输入更多，因为它包含所有 agent 的局部信息、所有 agent 的负荷和 PV 序列，以及联合动作。

12 项目中的动作后处理

12.1 为什么 raw action 不能直接执行

actor 输出的是 raw action，范围在 $[-1, 1]$ 。但是物理系统有很多边界：

- 电池 SoC 不能超过上下限；
- PV 弃光不能超过当前可用 PV；
- EV 不在家不能充电；
- safety projection 可能还要为了电网安全修正动作。

因此项目动作流程是：

raw action $\rightarrow$ SoC 可行映射 $\rightarrow$ 可选 safety projection $\rightarrow$ 本地可行性裁剪 $\rightarrow$ 环境执行.

(42)

12.2 电池动作映射

项目先根据当前 SoC 计算电池本步可行功率区间：

$$P_{i,t}^b \in [\underline{P}_{i,t}^b, \overline{P}_{i,t}^b]. \quad (43)$$

然后把 raw battery action 从 $[-1, 1]$ 映射到该区间：

$$P_{i,t}^b = \underline{P}_{i,t}^b + \frac{a_{i,t}^{raw,bat} + 1}{2} (\overline{P}_{i,t}^b - \underline{P}_{i,t}^b). \quad (44)$$

这样 actor 即使输出 1 或 -1，也不会直接把电池推到 SoC 越界。

13 PV 在项目中如何处理

13.1 PV 是预测输入，不是 actor 创造出来的

PV 可用功率来自数据和预测：

$$G_{i,t}. \quad (45)$$

actor 不能改变太阳真实发多少电。它只能决定使用多少 PV，弃掉多少 PV。

因此有两个量：

- $G_{i,t}$ ：可用 PV；
- $G_{i,t}^{use}$ ：实际利用 PV。

13.2 PV action 到利用率

actor 输出：

$$a_{i,t}^{pv} \in [-1, 1]. \quad (46)$$

环境把它映射成 PV 利用率：

$$\lambda_{i,t}^{pv} = \text{clip} \left(\frac{a_{i,t}^{pv} + 1}{2}, 0, 1 \right). \quad (47)$$

实际使用 PV：

$$G_{i,t}^{use} = \text{clip} (G_{i,t} \lambda_{i,t}^{pv}, 0, G_{i,t}). \quad (48)$$

弃光：

$$G_{i,t}^{curt} = G_{i,t} - G_{i,t}^{use}. \quad (49)$$

如果 $a^{pv} = 1$ ，PV 全部使用。如果 $a^{pv} = -1$ ，PV 全部弃掉。

13.3 PV 如何影响净负荷

净负荷是：

$$P_{i,t}^{net} = L_{i,t} - G_{i,t}^{use} + P_{i,t}^b + P_{i,t}^{ev}. \quad (50)$$

所以 PV 使用越多，净负荷越低；PV 弃光越多，净负荷越高：

$$G_{i,t}^{curt} \uparrow \Rightarrow G_{i,t}^{use} \downarrow \Rightarrow P_{i,t}^{net} \uparrow. \quad (51)$$

这也是 safety projection 能通过 PV 弃光改善电网安全的原因。例如 PV 过多可能造成反向潮流或电压升高，增加弃光可以降低注入。

13.4 PV 有没有直接 reward 惩罚

当前项目中没有一个显式的“弃光成本”主项。PV 主要通过净负荷影响潮流，再影响安全 penalty：

$$G^{curt} \rightarrow P^{net} \rightarrow \text{pandapower 潮流} \rightarrow L^v, L^\ell, L^{tr}. \quad (52)$$

所以 PV 的价值更多是间接体现：它改变电网状态和电池/EV 调度机会。

14 项目 reward function 逐项解释

14.1 总公式

代码在 envs/grid_env.py 的 GridEnv.step 中计算 reward。总公式可以写成：

$$\begin{aligned} r_{i,t} = & R_{i,t}^{stor} - C_{i,t}^{ev} - L_{i,t}^{act} - L_{i,t}^{soc} - L_{i,t}^{ev,soc} \\ & - L_{i,t}^{ev,dep} - L_{i,t}^{ev,over} - L_{i,t}^{ev,prog} - L_{i,t}^{ev,price} \\ & - L_{i,t}^{ev,proj} - L_{i,t}^{ev,eme} + B_{i,t}^{through} \\ & - L_{i,t}^v - L_{i,t}^\ell - L_{i,t}^{tr}. \end{aligned} \quad (53)$$

下面逐项解释。

14.2 电池收益

电池功率 $P_{i,t}^b$ 正数表示充电，负数表示放电。代码中的电池收益为：

$$R_{i,t}^{stor} = [\max(-P_{i,t}^b, 0) - \max(P_{i,t}^b, 0)] \tilde{p}_t \Delta t. \quad (54)$$

如果电池放电， $P^b < 0$ ，第一项为正，表示减少购电或获得等价收益。如果电池充电，第二项为正，但前面有负号，表示成本。

14.3 EV 充电成本

EV 充电成本为：

$$C_{i,t}^{ev} = w_{cost}^{ev} P_{i,t}^{ev} \tilde{p}_t \Delta t. \quad (55)$$

它在 reward 中是负项。EV 充得越多、价格越高，成本越大。

14.4 动作边界惩罚

如果电池已经接近 SoC 边界，但 actor 仍然推动它继续往边界方向走，会触发动作边界惩罚：

$$L_{i,t}^{act} = w^{act} |P_{i,t}^b| \mathbb{I}_{boundary}. \quad (56)$$

这里 $\mathbb{I}_{boundary}$ 是指示函数，表示是否在边界附近还往错误方向推。

14.5 SoC 正则惩罚

项目还设置了软 SoC 区间。如果电池 SoC 太接近硬边界，会有正则惩罚：

$$L_{i,t}^{soc} = w^{soc} \left[\max(0, SOC_{low}^{soft} - SOC_{i,t}^b)^2 + \max(0, SOC_{i,t}^b - SOC_{high}^{soft})^2 \right]. \quad (57)$$

物理意义是：不要总贴着电池边界运行，留一点调度余量。

14.6 EV SoC、离家、进度和价格惩罚

EV 相关惩罚包括：

- $L^{ev,soc}$ ：EV SoC 超出软范围的正则；
- $L^{ev,dep}$ ：离家时没有达到目标 SoC 的缺口惩罚；
- $L^{ev,over}$ ：hard 模式下离家超过目标过多的惩罚；
- $L^{ev,prog}$ ：soft 模式下连接期间没有跟上充电进度的惩罚；
- $L^{ev,price}$ ：在高于价格阈值时充电的价格感知惩罚；
- $L^{ev,proj}$ ：hard projection 额外抬高 EV 充电功率的惩罚；
- $L^{ev,eme}$ ：emergency charging 增加功率的惩罚。

离家缺口惩罚的典型形式是：

$$L_{i,t}^{ev,dep} = w_{dep}^{ev} \left[\max(0, SOC_{req}^{ev} - SOC_{i,t+1}^{ev}) \right]^2, \quad (58)$$

只在 departure step 触发。

进度目标可以理解为：从到家 SoC 到离家目标 SoC 之间画一条随时间上升的线。如果 EV 连接期间落后太多，就提前给惩罚，而不是只在离家那一刻才扣分。

14.7 throughput bonus

项目有一个电池吞吐 bonus:

$$B_{i,t}^{through} = w_t^{through} |P_{i,t}^b| \Delta t. \quad (59)$$

它在训练早期鼓励电池多探索充放电行为。权重会随训练进度下降，所以后期不会一直鼓励无意义动作。

14.8 电网安全惩罚

环境执行动作后，pandapower 计算潮流。若电压、线路或变压器越界，会产生安全惩罚:

$$L_{i,t}^{safe} = L_{i,t}^v + L_{i,t}^\ell + L_{i,t}^{tr}. \quad (60)$$

三项分别对应:

- L^v : 电压越界惩罚;
- L^ℓ : 线路负载率越界惩罚;
- L^{tr} : 变压器负载率越界惩罚。

三个 MADRL 方案的安全权重不同:

方案	安全机制	含义
MADRL_BASE	安全权重为 0, 无 projection	主要学习经济调度
MADRL_PENALTY	有安全 reward penalty, 无 projection	unsafe 后扣分, 让 actor 自己学会避免
MADRL_PROJECTION	有安全 penalty, 且启用 projection	执行前修正动作, 同时仍保留安全扣分信号

15 项目训练流程

15.1 整体伪代码

Listing 1: 项目 MADRL 训练流程

```

初始化每个 agent 的 actor、critic、target actor、target critic
初始化 replay buffer
初始化并行训练环境 SubprocVecEnv

```

```
while 已完成 episode 数 < 目标 episode 数:
    1. 每个 actor 根据当前 obs 输出 raw action
    2. 给 raw action 加探索噪声
    3. 把电池动作映射到 SoC 可行功率区间
    4. 以一定概率进行随机可行电池探索
    5. 如果是 projection 方案, 调用 JointGridSafetyProjector
    6. 再做本地可行性裁剪
    7. 环境执行 action, 返回 reward、next_obs、done、info
    8. transition 写入 ReplayBuffer
    9. buffer 足够大后随机采样 batch
    10. 对每个 agent 更新 critic
    11. 每隔 policy_update_freq 次更新 actor
    12. soft update target networks
```

保存模型、meta 信息、训练 reward 曲线

15.2 探索噪声

训练时 actor 输出后会加高斯噪声:

$$a^{raw} \leftarrow \text{clip}(a^{raw} + \epsilon, -1, 1). \quad (61)$$

噪声标准差从初始值逐渐衰减到最小值。训练早期多试错, 训练后期更相信学到的 actor。

项目还会以一定概率随机采样可行电池功率。这个概率也会逐渐下降。它帮助电池动作空间在训练早期被充分探索。

15.3 Critic 更新流程

对第 i 个 agent:

1. target actors 对 next observation 生成下一步动作;
2. 给下一步动作加 target smoothing 噪声;
3. 对下一步动作做同样的后处理和 projection;
4. target twin critics 计算两个 Q 值;
5. 取较小值构造 Bellman target;
6. 当前 critic 对 batch 中真实动作计算 Q;
7. 用 MSE 更新 critic。

公式:

$$y_i = R_{i,t}^{(n)} + \Gamma_{i,t}^{(n)} \min(Q_{i,1}^{target}, Q_{i,2}^{target}). \quad (62)$$

loss:

$$L_i^{critic} = \text{MSE}(Q_{i,1}, y_i) + \text{MSE}(Q_{i,2}, y_i). \quad (63)$$

15.4 Actor 更新流程

更新第 i 个 actor 时, 项目只替换联合动作中第 i 个 agent 的动作:

$$\mathbf{a}_t^{policy} = (a_{1,t}, \dots, \pi_i(o_{i,t}), \dots, a_{N,t}). \quad (64)$$

然后仍然经过本地可行性处理和可选 projection, 再送入 critic:

$$L_i^{actor} = -\mathbb{E} \left[Q_{i,1}(\mathbf{o}_t, \mathbf{a}_t^{policy}) \right]. \quad (65)$$

这相当于问 critic: 如果只把 agent i 的动作换成 actor 当前想做的动作, 长期价值会不会更好?

16 Safety penalty 和 safety projection

16.1 Safety penalty: 事后扣分

Safety penalty 是动作执行之后才发生的。流程是:

$$\text{执行动作} \rightarrow \text{pandapower 潮流} \rightarrow \text{发现越界} \rightarrow \text{reward 扣分}. \quad (66)$$

它像考试后扣分: 能告诉 actor 哪些行为不好, 但不能阻止这一步 unsafe action 已经发生。

16.2 Safety projection: 执行前修正

Safety projection 是动作执行前发生的:

$$\mathbf{a}^{raw} \rightarrow \mathbf{a}^{projected} \rightarrow \text{环境执行}. \quad (67)$$

它像安全过滤器。如果 actor 输出动作可能导致电压、线路或变压器风险, projection 会尽量把动作拉回近似安全区域。

17 Safety projection 如何实现

17.1 投影变量

JointGridSafetyProjector 只投影两类变量:

$$\mathbf{x} = [P_1^b, \dots, P_N^b, G_1^{curt}, \dots, G_N^{curt}]. \quad (68)$$

它不投影 EV 充电。EV 充电由 EV hard projection 或 emergency 逻辑单独处理。

17.2 为什么电池和 PV 弃光可以一起投影

把 PV 使用写成 $G^{use} = G - G^{curt}$, 净负荷为:

$$P_i^{net} = L_i - G_i + G_i^{curt} + P_i^b + P_i^{ev}. \quad (69)$$

可见:

$$\frac{\partial P_i^{net}}{\partial P_i^b} = 1, \quad \frac{\partial P_i^{net}}{\partial G_i^{curt}} = 1. \quad (70)$$

也就是说, 电池多充 1 kW 和 PV 多弃 1 kW 都会让该节点净负荷增加 1 kW。projection 因此可以把它们放进同一个变量向量。

17.3 灵敏度矩阵

真实 AC 潮流是非线性的。项目用有限差分得到线性灵敏度近似。先在零注入附近运行潮流得到基准:

$$\mathbf{v}^{base}, \quad \boldsymbol{\ell}^{base}. \quad (71)$$

对第 j 个 agent 做小扰动 $\delta = 0.25$ kW:

$$\mathbf{v}^{+j} = \text{PF}(+\delta \mathbf{e}_j), \quad \mathbf{v}^{-j} = \text{PF}(-\delta \mathbf{e}_j). \quad (72)$$

电压灵敏度为:

$$S_{:,j}^v = \frac{\mathbf{v}^{+j} - \mathbf{v}^{-j}}{2\delta}. \quad (73)$$

线路负载率灵敏度为:

$$S_{:,j}^\ell = \frac{\boldsymbol{\ell}^{+j} - \boldsymbol{\ell}^{-j}}{2\delta}. \quad (74)$$

17.4 基础净负荷

投影时, EV 先当作固定负荷:

$$P_i^{base} = L_i - G_i + P_i^{ev}. \quad (75)$$

投影变量改变净负荷:

$$P_i^{net} = P_i^{base} + P_i^b + G_i^{curt}. \quad (76)$$

17.5 约束写成 half-space

线性近似下，电压、线路、变压器约束都可以整理成：

$$\mathbf{r}_m^\top \mathbf{x} \leq b_m. \quad (77)$$

所有约束的交集构成近似安全集合：

$$\mathcal{C} = \{\mathbf{x} : \mathbf{r}_m^\top \mathbf{x} \leq b_m, \forall m\}. \quad (78)$$

17.6 逐行投影公式

如果某一行约束没有违反，即：

$$\mathbf{r}^\top \mathbf{x} \leq b, \quad (79)$$

则不修改。

如果违反了，就把 $\mathbf{x}$ 沿着约束法向量拉回边界：

$$\mathbf{x} \leftarrow \mathbf{x} - \frac{\max(0, \mathbf{r}^\top \mathbf{x} - b)}{\|\mathbf{r}\|_2^2} \mathbf{r}. \quad (80)$$

项目会对所有约束行循环投影，并重复：

$$\text{projection_iters} = 3 \quad (81)$$

轮。每轮之后还会裁剪本地物理范围：

$$P_i^b \leftarrow \text{clip}(P_i^b, \underline{P}_i^b, \overline{P}_i^b), \quad (82)$$

$$G_i^{\text{curt}} \leftarrow \text{clip}(G_i^{\text{curt}}, 0, G_i). \quad (83)$$

17.7 投影后的动作转回 actor 格式

投影得到的是物理量 P_i^b 和 G_i^{curt} ，最后要转回环境动作：

$$a_i^{\text{bat}} = \frac{P_i^b}{P_i^{b, \max}}, \quad (84)$$

$$a_i^{\text{pv}} = 2 \left(1 - \frac{G_i^{\text{curt}}}{G_i} \right) - 1. \quad (85)$$

EV action 保持原值。

17.8 projection 的代价

projection 会提高安全性，但也有副作用：

$$\mathbf{a}^{\text{actor}} \neq \mathbf{a}^{\text{executed}}. \quad (86)$$

actor 想做的动作和实际执行动作不完全一样，学习会更复杂。实验上常见现象是：projection 方案更安全，但经济性可能下降，因为部分套利动作会被安全过滤器修正。

18 执行阶段：训练好以后用什么

训练结束后，模型保存 actor 和 critic 参数。但实际评估或部署时，主要使用 actor：

$$o_{i,t} \rightarrow \pi_i(o_{i,t}) \rightarrow \text{postprocess} \rightarrow d_{i,t}^{exec}. \quad (87)$$

critic、replay buffer、Bellman target 都是训练时使用的工具。执行阶段不再更新网络，也不需要 replay buffer。

如果模型 meta 中 `projection=True`，评估控制器会启用同样的 safety projection。代码上，就是 `MADRLController` 创建 `JointGridSafetyProjector` 后再执行动作后处理。

19 从代码角度看完整闭环

Listing 2: 项目中的 MADRL 闭环

```

1. GridEnv._obs 产生 observation:
    madrl_local, safety_local,
    wholesale_price_relative_seq,
    wholesale_price_spread_seq,
    load_seq, pv_seq

2. Actor 输出 raw action:
    raw = actor(obs_i)

3. 训练时加入探索:
    raw = clip(raw + noise, -1, 1)

4. 动作后处理:
    map_actor_output_to_soc_feasible_action
    optional JointGridSafetyProjector
    enforce_local_action_feasibility

5. GridEnv.step(action):
    EV hard/emergency 逻辑
    PV effective = pv * utilization
    net = load - pv_effective + battery_kw + ev_charge_kw
    pandapower power flow
    reward = economic terms - penalties + bonus

6. ReplayBuffer.add_batch:
    形成 n-step transition

7. 训练:

```

sample batch

critic 学 Bellman target

actor 最大化 critic Q

soft update target networks

20 常见名词小词典

名词	解释
agent	做决策的主体，本项目中是每个 prosumer 控制器
environment	智能体交互的系统，本项目中是配电网仿真环境
state	环境完整真实情况，实际控制器不一定全知道
observation	agent 能看到并输入网络的信息
action	actor 输出的控制动作，本项目是电池、PV、EV 三维动作
reward	环境给动作的评分，用来引导学习
return	从当前开始的未来折扣累计奖励
policy	策略，从观测到动作的映射
actor	策略网络，负责输出动作
critic	价值网络，负责估计 Q 值
Q 值	当前情况下做某动作后，未来累计 reward 的估计
Bellman target	用当前 reward 加未来 Q 估计构造出来的 critic 训练目标
bootstrap	不把所有未来都展开，而是用 critic 对远期未来进行估计
replay buffer	存经验的池子，训练时随机抽样
n-step return	先累加未来 n 步真实 reward，再接上 bootstrap Q 值
target network	慢速更新的目标网络，用来稳定 Bellman target
twin critic	两个 critic，同一 target 取较小 Q 值以减少过高估计
TD3	连续动作 actor-critic 算法，使用 twin critic、target smoothing、delayed update
MATD3	TD3 的多智能体版本，本项目 MADRL 主线接近 MATD3
CTDE	集中训练、分散执行
safety penalty	动作执行后根据越界情况扣 reward
safety projection	动作执行前把电池功率和 PV 弃光投影到近似安全集合

21 总结

本项目中的 MADRL 可以概括为：

每个 agent 用自己的 actor 根据局部观测输出动作；训练时每个 critic 看联合观测和联合动作，学习 Bellman target；算法采用 MATD3 风格的 twin critic、target network、n-step replay buffer、target policy smoothing 和 delayed actor update；reward function 同时考虑经济性、EV 需求、电池边界和电网安全；projection 方案在动作执行前修正电池功率和 PV 弃光。

如果只记一条主线，可以记：

actor 做动作 → 环境给 reward → critic 学 Q 值 → actor 学会做 critic 认为更好的动作.
(88)

Bellman target 的核心也可以记成一句话：

当前动作的目标价值 = 已经拿到的短期真实 reward + 对剩余未来的 target critic 估计。

PV 在项目中不是 actor 创造的电源，而是预测给出的可用发电；actor 通过 PV action 决定利用率，projection 通过调整 PV 弃光和电池功率来改善电网安全。这样，MADRL 不只是学省钱，还可以在 reward penalty 和 projection 的帮助下学习更安全的分布式调度。